

DIGITAL VLSI DESIGN

Combinational Circuits

50 Advanced Questions

VLSI Selection Level

With Key Concept Hints

Topic	Subject Area	Coverage
Q 1–10	Adders & Subtractors	Half Adder, Full Adder, RCA, CLA, CLA hierarchy, BCD addition, CSA, prefix adders
Q 11–20	Multipliers & Dividers	Array multiplier, Booth encoding, Wallace Tree, Non-restoring/SRT division, squaring, SD arithmetic
Q 21–30	Encoders & Decoders	3-to-8 decoder, priority encoder, Gray code, BCD-7seg, XS3, comparator, Hamming, dual-rail
Q 31–40	MUX / DEMUX	Shannon expansion, 32:1 MUX tree, LUT/FPGA, barrel shifter, priority MUX, TG vs. gate MUX
Q 41–46	Comparators & Parity	Parity generator, Hamming(7,4), magnitude comparator, wired-OR, BCD detect, Manchester carry, hazards
Q 47–50	Advanced Topics	Multi-level logic, equivalence checking, PTL XOR, Boolean network DAG, stuck-at faults, Quine-McCluskey

Each question is crafted to probe depth of understanding beyond textbook recall — covering gate-level tradeoffs, transistor-level implications, delay analysis, area-power-speed triangles, and CAD tool relevance. Expand key concept hints only after attempting each question independently.

SECTION 1

Adders & Subtractors

Q1–Q10

Q1

A Half Adder is implemented using one XOR and one AND gate. If both inputs are 1, what are the Sum and Carry outputs? Extend this analysis: can a Half Adder detect an overflow in 1-bit signed arithmetic, and why or why not? What structural modification would be required to handle signed overflow?

KEY CONCEPT / APPROACH HINT

Sum=XOR=0, Carry=AND=1. In 1-bit signed 2's complement, +1+1 overflows the representation. The carry IS produced but HA cannot distinguish arithmetic overflow from unsigned carry-out, nor can it produce a sign-extended result. A Full Adder with sign extension logic is required.

Q2

Derive the complete Boolean expressions for Sum and Carry-out of a Full Adder with inputs A, B, Cin. Then implement a Full Adder using exactly two Half Adders and one OR gate, justifying each gate's role with a logical proof.

KEY CONCEPT / APPROACH HINT

Sum=A XOR B XOR Cin; Cout=AB+Cin(A XOR B). HA1: Sum1=A XOR B, C1=AB. HA2: Sum=Sum1 XOR Cin, C2=Cin.Sum1. OR: Cout=C1|C2. Proof: C1 and C2 can never be simultaneously 1, so OR suffices.

Q3

A 4-bit Ripple Carry Adder (RCA) adds A=1111 and B=0001. Trace carry propagation bit-by-bit and compute the total critical path delay if each Full Adder has 2ns carry delay and 3ns sum delay. Derive the general formula for n-bit RCA worst-case delay.

KEY CONCEPT / APPROACH HINT

Carries ripple C0->C1->C2->C3->C4 all generating. Critical path=(3 carry stages)+(1 sum stage)=(3x2)+3=9ns. General: $T_{RCA} = (n-1)*t_{carry} + t_{sum}$. For n=4: $3*2+3=9ns$ confirmed.

Q4

Explain why the Carry Lookahead Adder (CLA) is faster than RCA. Define the Generate (G_i) and Propagate (P_i) terms for each bit stage and write the fully expanded Boolean expression for C₄ in a 4-bit CLA showing all terms explicitly.

KEY CONCEPT / APPROACH HINT

$G_i = A_i \cdot B_i$, $P_i = A_i \oplus B_i$. $C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$. All carries computed in parallel in 2 gate delays -- $O(\log n)$ vs $O(n)$ for RCA. No carry chain dependency.

Q5

Differentiate between a Full Subtractor and Full Adder structurally. A 4-bit subtractor performs $A-B$ using 2's complement with $A=0011$ and $B=0111$. Trace the complete operation, identify borrow generation, and express the final result interpreting the 2's complement encoding.

KEY CONCEPT / APPROACH HINT

Full Subtractor: $\text{Diff} = A \oplus B \oplus \text{Bin}$, $\text{Bout} = A'B + A'\text{Bin} + B\text{Bin}$. 2's comp: complement $B=1000$, add 1 $\rightarrow 1001$. $A+B'+1=0011+1001=1100$, no final carry $\rightarrow$ borrow generated $\rightarrow$ negative result. 1100 in 2's comp = -4. Verified: $3-7=-4$.

Q6

Design a shared circuit that simultaneously computes both sum and difference of two 4-bit numbers using a single Full Adder array. Specify the additional logic, draw the data-path, and calculate the exact gate overhead introduced.

KEY CONCEPT / APPROACH HINT

Add mode-select line M . XOR each B_i with M (conditional invert). Feed M as C_{in} . When $M=0$: $A+B$ (addition). When $M=1$: $A+B'+1=A-B$ (subtraction). Overhead: exactly 4 XOR gates + 1 wire ($C_{in}=M$). The 4 FAs are shared completely.

Q7

Explain the Carry Save Adder (CSA) principle. Show how three 4-bit numbers A , B , C are reduced to two output vectors, and explain the critical role of CSAs in the partial product reduction phase of high-speed multipliers.

KEY CONCEPT / APPROACH HINT

CSA: 3 inputs per bit $\rightarrow$ (sum, carry) pair with NO carry propagation. Outputs: Sum vector S and Carry vector C (shifted left 1). Three n -bit numbers reduced to two n -bit numbers in $O(1)$ delay. In Wallace Tree: multiple CSA stages reduce n partial products to 2 rows in $O(\log_{1.5} n)$ stages before final CPA.

Q8

Compare area, delay, and power tradeoffs across four adder architectures: Ripple Carry, Carry Lookahead, Carry Select, and Kogge-Stone Prefix adder. Provide the asymptotic complexity for each and specify the VLSI conditions under which each architecture is optimal.

KEY CONCEPT / APPROACH HINT

RCA: $O(n)$ delay, $O(n)$ area, lowest power. CLA: $O(\log n)$ delay with fan-in limitation, $O(n \log n)$ area. Carry Select: $\sqrt{n}$ delay, 2x area of RCA. Kogge-Stone: $O(\log n)$ delay, $O(n \log n)$ area, high fan-out/wiring. Choose: RCA for low-power IoT, Kogge-Stone for GHz processors, Carry Select for balanced chips.

Q9

In BCD addition, describe precisely the correction step applied after binary addition and the condition triggering it. Show step-by-step for the addition of BCD digits 5 (0101) + 8 (1000), including carry generation.

KEY CONCEPT / APPROACH HINT

Binary $0101 + 1000 = 1101 = 13d$. Since $13 > 9$, correction needed: add 0110. $1101 + 0110 = 10011 \rightarrow$ result is 0011 with BCD carry=1 $\rightarrow$ BCD result: 1|0011 = 13 in BCD. Rule: if binary sum > 9 (1001) OR carry generated, add 6 and propagate carry to next BCD position.

Q10

A 16-bit adder must meet minimum worst-case delay. Compare a flat 16-bit CLA against a two-level hierarchical 4x4 CLA in terms of gate delays and fan-in constraints. Write the group Generate and group Propagate expressions for the top-level CLA block.

KEY CONCEPT / APPROACH HINT

Flat 16-bit CLA: fan-in=16 for top carries -- physically impractical. Hierarchical 4x4: each 4-bit block computes G_{block} , P_{block} in 2 delays; top CLA computes group carries in 2 more; sum in 1 more: total 5 delay levels with max fan-in=4. $G_{\text{block}_i} = G_{i3} + P_{i3}.G_{i2} + P_{i3}.P_{i2}.G_{i1} + P_{i3}.P_{i2}.P_{i1}.G_{i0}$.

SECTION 2

Multipliers & Dividers

Q11–Q20

Q11

Design a 4x4 unsigned binary multiplier using AND gates and Full Adders. Write all partial products for A=1011 and B=1101 and show the complete summation array layout, identifying the role of each row of adders.

KEY CONCEPT / APPROACH HINT

16 AND gates for all partial products. PP0=1011x1=01011, PP1=1011x0 shifted=00000x, PP2=1011x1 shifted=101100, PP3=1011x1 shifted=1011000. Sum: 0001011+000000+101100+1011000=10001111=143. Array: 3 rows of adders, each row sums one new partial product with running total.

Q12

Explain Radix-2 Booth encoding in binary multiplication. Demonstrate the complete algorithm for multiplying +6 (0110) by -3 (1101 in 2's complement) in a 4-bit signed system, showing all Booth steps and partial product accumulation.

KEY CONCEPT / APPROACH HINT

Booth examines bit pairs $[b_i, b_{i-1}]$: 00=0, 01=+M, 10=-M, 11=0. Append 0: -3=11010. Pairs from LSB: b1b0=10->-M=-6, b2b1=11->0, b3b2=01->+M=+6 shifted by 2=24, b4b3=11->0. Sum: -6+0+24+0=18. But 4-bit signed: use sign-extended 8-bit. $6 \times (-3) = -18$. Verify sign extension applied correctly.

Q13

Explain Wallace Tree multiplier architecture with a block diagram description. Derive the delay complexity compared to an array multiplier and calculate the exact number of CSA stages required for an 8x8 multiplier.

KEY CONCEPT / APPROACH HINT

Array multiplier: $O(n)$ delay from sequential row additions. Wallace Tree: each CSA stage reduces row count by factor 1.5 (3 rows -> 2). For 8 partial products: need $\text{ceil}(\log_{1.5}(8)) = \text{ceil}(4.42) = 5$ CSA stages then one final CPA. Delay = $O(\log n)$ vs $O(n)$. For 8-bit: 5 CSA stages + 1 16-bit adder.

Q14

Trace a non-restoring division algorithm for $8/3$. Show the first 4 iterations with partial remainder and quotient bit generated at each step. Explicitly contrast with restoring division in terms of hardware operations and cycle count.

KEY CONCEPT / APPROACH HINT

Divisor $D=3$. Start $R=0$. Shift dividend: $R=8$. Step1: $R=8 \geq 0$, subtract: $R=8-3=5$, $q_bit=1$. Step2: $2R=10 \geq 0$, $R=10-3=7$, $q=1$. Step3: $2R=14 \geq 0$, $R=14-3=11$, $q=1$. Step4: $2R=22 \geq 0$, $R=22-3=19...$
Non-restoring: no restoration step needed when $R < 0$, just add next cycle. Saves one add/sub per negative remainder vs restoring.

Q15

Explain the SRT division algorithm principle and the significance of Radix-4 SRT. Describe the Prescaled Divisor (PD) lookup table structure and explain the historical significance of the Intel Pentium FDIV bug in the context of this algorithm.

KEY CONCEPT / APPROACH HINT

SRT: Sweeney-Robertson-Tocher uses redundant quotient digits to avoid full subtraction -- digit selected from small table based on partial remainder MSBs. Radix-4 SRT: 2 bits/cycle using digits $\{-2, -1, 0, 1, 2\}$, requires 5-bit partial remainder lookup. Pentium bug: missing 5 entries in 1066-entry PD table caused FDIV errors for specific operand patterns -- discovered 1994.

Q16

Design a combinational squaring circuit for a 4-bit unsigned input exploiting the mathematical symmetry $P_{ij} = P_{ji}$ in squaring. Quantify exactly how many AND gates and partial product bits are saved compared to a general 4x4 multiplier.

KEY CONCEPT / APPROACH HINT

General 4x4: 16 AND gates, 16 partial products. Squaring: diagonal $P_{ii} = A_i$ (no AND needed -- reduced to wire). Off-diagonal: $P_{ij}=P_{ji}$, computed once and doubled (left-shift). Unique products: $n + n(n-1)/2 = 4+6=10$ AND gates needed (vs 16). Save 6 AND gates and approximately 6 partial product rows. Total: 10 AND gates.

Q17

Implement a combinational absolute value circuit for an 8-bit 2's complement signed number without using a subtractor or comparator. Derive the complete logic, specify all gates, and prove correctness for both positive and negative inputs.

KEY CONCEPT / APPROACH HINT

Principle: if MSB=0, output=input. If MSB=1, output=2's complement=invert all bits + 1. Implementation: XOR each bit with MSB (conditional bitwise inversion), then add MSB as carry-in to ripple-carry incrementer. Gates: 8 XOR gates + 7 half-adders (incrementer). Proof: MSB=0 -> XOR gives identity -> +0=same. MSB=1 -> invert+1=2's complement.

Q18

In VLSI high-performance multiplier design, justify why a Kogge-Stone prefix adder is the preferred Final Carry Propagate Adder (CPA). Compare it to Brent-Kung and Ladner-Fischer prefix adders in terms of depth, fanout, and wire complexity.

KEY CONCEPT / APPROACH HINT

Kogge-Stone: $O(\log n)$ depth, max fanout=2, regular structure, all prefix nodes computed -- fastest but highest wire count $O(n \log n)$. Brent-Kung: $O(\log n)$ depth but 2x passes needed, lower wire count, higher fanout at top. Ladner-Fischer: compromise. For GHz-class multipliers, Kogge-Stone is optimal: minimum depth matches critical path requirement and regular layout aids place-and-route.

Q19

A 32-bit multiplier uses a 4:2 compressor tree instead of a 3:2 CSA tree. Define a 4:2 compressor precisely, specify its transistor implementation in CMOS, and quantify the improvement in partial product reduction levels for 32 partial products.

KEY CONCEPT / APPROACH HINT

4:2 compressor: 4 data inputs + 1 carry-in -> 1 sum output + 2 carry outputs. Built from 2 cascaded FAs (or optimized 6-XOR custom cell). Reduces 4 rows to 2 in same delay as 3:2 reduces 3 to 2. For 32 partial products: 3:2 CSA tree needs $\lceil \log_{1.5}(32) \rceil = 9$ stages; 4:2 tree needs $\lceil \log_2(32) \rceil = 5$ stages. Saves 4 reduction levels.

Q20

Explain carry-free addition in a Signed Digit (SD) number system. Show that carry propagation is limited to one position, derive an example of SD addition, and discuss the primary VLSI implementation challenges including encoding overhead and conversion cost.

KEY CONCEPT / APPROACH HINT

SD digits: $\{-1, 0, +1\}$. Addition: each digit position produces intermediate sum in $\{-2, -1, 0, 1, 2\}$ -- split into carry in $\{-1, 0, 1\}$ to next position and local digit in $\{-1, 0, 1\}$. Carry propagates at most ONE position -- $O(1)$ delay regardless of word length. Encoding: 2 bits per digit (00=-1, 01=0, 10=+1). VLSI challenges: 2x bit-width, final conversion to 2's complement requires $O(\log n)$ adder, complex SD-to-binary interface.

SECTION 3

Encoders & Decoders

Q21–Q
30

Q21

Design a 3-to-8 decoder with active-high outputs. Write the Boolean expression for each of the 8 outputs, implement the circuit using only 2-input NAND gates, and determine the minimum gate count required with justification.

KEY CONCEPT / APPROACH HINT

Y_i = product of minterm inputs for index i . Example: $Y_0 = A'B'C'$, $Y_7 = ABC$. NAND-NAND implementation: each minterm needs a 3-input AND $\rightarrow$ implemented as 2 two-input NANDs + 1 NAND for inversion = 3 NANDs per output. 8 outputs: 24 NANDs minimum before input literal sharing. With sharing of complemented inputs (3 inverters): reduce to 8 three-input NANDs + 3 inverters (in NAND form).

Q22

A 4-to-16 decoder is constructed using two 3-to-8 decoders and minimal additional logic. Draw and describe the complete block diagram, explain the enable signal routing, and calculate the total gate count overhead beyond the two sub-decoders.

KEY CONCEPT / APPROACH HINT

MSB A_3 selects between decoders: $A_3=0$ enables upper decoder (Y_0 - Y_7), $A_3=1$ enables lower (Y_8 - Y_{15}). A_0 - A_2 connect identically to both decoders' address inputs. Enable logic: $\text{top_EN} = A_3'$, $\text{bottom_EN} = A_3$. Overhead: 1 inverter only. Both decoders share A_0 - A_2 inputs. This architecture scales to any 2^n decoder using $\log n$ levels of decoder hierarchy.

Q23

Implement a 4-bit binary-to-Gray code converter and its inverse (Gray-to-binary). Derive Boolean expressions for all bits, implement with minimum XOR gates, and explain with two concrete examples why Gray code is essential in rotary encoders and ADC designs.

KEY CONCEPT / APPROACH HINT

$B \rightarrow G$: $G_3 = B_3$, $G_2 = B_3 \text{ XOR } B_2$, $G_1 = B_2 \text{ XOR } B_1$, $G_0 = B_1 \text{ XOR } B_0$. $G \rightarrow B$: $B_3 = G_3$, $B_2 = B_3 \text{ XOR } G_2$, $B_1 = B_2 \text{ XOR } G_1$, $B_0 = B_1 \text{ XOR } G_0$. Example: Binary 0111 $\rightarrow$ Gray 0100 (sequential XOR chain). Rotary encoder: only 1 bit changes per position step $\rightarrow$ no transient multi-bit errors if encoder is between positions. ADC: eliminates metastability glitches in comparator arrays during conversion.

Q24

Design the BCD-to-7-segment decoder for digit 6 specifically. Write the complete truth table entry, identify all don't-care conditions, and minimize the SOP expression for each of the 7 segments using Karnaugh maps showing all prime implicants.

KEY CONCEPT / APPROACH HINT

BCD 6=0110. Segments for '6' on standard display: a(top)=1, b(top-right)=0, c(bot-right)=1, d(bottom)=1, e(bot-left)=1, f(top-left)=1, g(middle)=1. Don't-cares: BCD inputs 1010-1111 (6 invalid codes). K-map with 6 don't-cares dramatically simplifies each segment expression. For segment b: it is OFF only for 6 (and some others) -- K-map gives $b = A + B'D' + B'C$ where A=MSB.

Q25

What is a priority encoder? Design a complete 8-input priority encoder with I7 as highest priority. Write the Boolean expressions for all three output bits (Y2,Y1,Y0) and the valid output V, using priority-masked logic rather than truth table lookup.

KEY CONCEPT / APPROACH HINT

$V = I7 \vee I6 \vee I4 \vee I3 \vee I2 \vee I1 \vee I0$. $Y2 = I7 \vee I6 \vee I5 \vee I4$. $Y1 = I7 \vee I6 \vee (\sim I7 \ \& \ \sim I6 \ \& \ (I3 \vee I2))$. $Y0 = I7 \vee (\sim I7 \ \& \ I5) \vee (\sim I7 \ \& \ \sim I6 \ \& \ \sim I5 \ \& \ \sim I4 \ \& \ I3) \vee (\sim I7 \ \& \ \sim I6 \ \& \ \sim I5 \ \& \ \sim I4 \ \& \ \sim I3 \ \& \ \sim I2 \ \& \ I1)$. Each lower-priority input gated by absence of all higher-priority inputs. V validates output when at least one input active.

Q26

Design an Excess-3 to BCD decoder. Derive the mathematical relationship between the two codes, construct the complete truth table with don't-care identification, and minimize each BCD output bit using algebraic methods rather than K-maps.

KEY CONCEPT / APPROACH HINT

Excess-3 = BCD + 3. Therefore $BCD = XS3 - 3 = XS3 + 13$ (add 1101 and discard carry, or equivalently subtract 0011). Valid XS3: 0011(0) to 1100(9). Invalid: 0000,0001,0010,1101,1110,1111 are don't-cares. Implement as 4-bit subtractor with constant B=0011 (using XOR-based subtraction: XOR with 0011 and add 0001). Each output simplified algebraically with don't-cares.

Q27

A cascadable 4-bit magnitude comparator generates $A > B$, $A = B$, $A < B$ outputs. Derive the Boolean logic for all three outputs, then show the complete cascade connection to build an 8-bit comparator, specifying exactly how cascade input ports are connected and why.

KEY CONCEPT / APPROACH HINT

$A = B$: $\text{XNOR}(A_3, B_3) \cdot \text{XNOR}(A_2, B_2) \cdot \text{XNOR}(A_1, B_1) \cdot \text{XNOR}(A_0, B_0)$. $A > B$: hierarchical masking from MSB down. Cascade: lower 4-bit comparator outputs (GT_low , EQ_low , LT_low) feed cascade inputs of upper comparator. Upper $A = B = \text{upper_EQ} \text{ AND } \text{EQ_low}$. Upper $A > B = \text{upper_GT} \text{ OR } (\text{upper_EQ} \text{ AND } \text{GT_low})$. Upper $A < B = \text{upper_LT} \text{ OR } (\text{upper_EQ} \text{ AND } \text{LT_low})$. This maintains full priority: upper bits override lower.

Q28

Explain the 2-of-5 code structure, prove it can detect all single-bit errors, demonstrate why it cannot correct single-bit errors, and compare its error detection capability to a simple parity code using the Hamming distance metric.

KEY CONCEPT / APPROACH HINT

2-of-5: exactly 2 bits set in 5. Valid codewords: $C(5,2)=10$ (sufficient for decimal digits 0-9). Minimum Hamming distance between any two valid codewords = 2 (e.g., 11000 vs 10100 differ in 2 positions). Detection: single-bit flip changes weight from 2 to 1 or 3 -- both invalid, always detectable. No correction: knowing one bit is wrong doesn't identify which of 4 possible bit positions to flip. Parity: distance=2 also, same detection power, half the redundancy.

Q29

Prove that a decoder can implement any arbitrary Boolean function of n variables. Using a 3-to-8 decoder with an external OR gate, implement $f(A,B,C) = \sum m(1,3,5,7)$ showing all connections. Explain how this principle forms the theoretical basis of ROM and PLA in VLSI.

KEY CONCEPT / APPROACH HINT

Proof: decoder activates exactly one minterm output for each input combination -- it generates the complete minterm basis. Any SOP is a subset of minterms -> select with OR gate. $f = m_1 + m_3 + m_5 + m_7 = Y_1 | Y_3 | Y_5 | Y_7$ (connect these 4 outputs to OR gate). Note: $f = C$ (all odd minterms). ROM: decoder is address decoder, memory bits select minterms for each function output. PLA: programmable AND plane=decoder, programmable OR plane=function selection.

Q30

Design a dual-rail encoder for a 4-input one-hot code and explain the dual-rail logic paradigm. Why is dual-rail encoding used in asynchronous VLSI circuits for completion detection, and how does it eliminate the timing hazards present in single-rail logic?

KEY CONCEPT / APPROACH HINT

Dual-rail: each signal has true wire T and complement wire F. For one-hot I0-I3: output pair (T,F) where $T=I_i$ selected, $F=\text{complement}$. Valid states: (1,0)=logic 1, (0,1)=logic 0. Spacer state: (0,0)=no data. Completion detection: valid data indicated by $(T \text{ OR } F)=1$ -- detectable without a clock. Eliminates hazards: transitions are monotonic (spacer->data->spacer), no glitches possible since both rails cannot simultaneously be 1 in a properly designed circuit.

SECTION 4

MUX / DEMUX & Barrel Shifter

Q31–Q40

Q31

A 2-to-1 MUX has inputs A, B and select S. Prove rigorously that any Boolean function of 2 variables can be implemented using a single 2-to-1 MUX with allowed constant inputs {0,1,S,S'}. As a specific case, implement $f=A \text{ XOR } B$ using only 2-to-1 MUXes.

KEY CONCEPT / APPROACH HINT

2:1 MUX: $f=S'I_0+S'I_1$. 4 functions of S possible per input combination: {0,1,S,S'}. With 2 inputs and 4 choices each: $4 \times 4 = 16$ configurations $\rightarrow$ covers all 16 Boolean functions of 2 variables. XOR: fix $S=A$. $f=A'B+A'B'$ needed. Set $I_0=B$ (when $A=0 \rightarrow f=B$), $I_1=B'$ (when $A=1 \rightarrow f=B'$). Requires one inverter for B'. Alternatively: cascade two 2:1 MUXes without inverter using SN identity.

Q32

State Shannon's expansion theorem formally and use it to implement $f(A,B,C) = \sum m(1,3,5,7)$ using a 4-to-1 MUX with A and B as select lines. Show all Shannon expansion steps explicitly.

KEY CONCEPT / APPROACH HINT

Shannon: $f(x_1, \dots, x_n) = x'_1 f(0, x_2, \dots, x_n) + x_1 f(1, x_2, \dots, x_n)$. Expand on A then B: $f(0,0,C)=C'C=?$ Compute: $m_1=A'B'C$, $m_3=A'BC$, $m_5=AB'C$, $m_7=ABC$. $f(0,0,C)=m_1=C$. $f(0,1,C)=m_3=C$. $f(1,0,C)=m_5=C$. $f(1,1,C)=m_7=C$. MUX: $I_0=I_1=I_2=I_3=C$. Select: $S_1=A$, $S_0=B$. Result: $f=C$ regardless of A,B, confirming $f(A,B,C)=C$.

Q33

Implement a 32-to-1 MUX using only 4-to-1 MUX components. Derive the exact number of 4:1 MUXes required, draw the complete 3-level hierarchy, and show precisely how the 5 select lines are distributed across the levels.

KEY CONCEPT / APPROACH HINT

Need to select 1 of $32=2^5$ inputs using 5 select bits. Level 1: 8 four-to-one MUXes, each selecting 1 of 4 inputs, controlled by $S[1:0]$. 8 outputs. Level 2: 2 four-to-one MUXes, each selecting 1 of 4 (from level 1 outputs), controlled by $S[3:2]$. 2 outputs. Level 3: 1 two-to-one MUX... but must use 4:1 only: pad level 2 to 4 inputs with 2 repeats and use 1 final 4:1 with $S[4:4]$ and fixed inputs. Total: $8+2+1=11$ four-to-one MUXes. Alternatively use true 2-of-5 selection.

Q34

A 1-to-8 DEMUX distributes input D to one of 8 outputs based on 3-bit address. Write the Boolean expression for each output, compare functionally and structurally with a 3-to-8 decoder, and explain how the enable line of a decoder implements DEMUX behaviour.

KEY CONCEPT / APPROACH HINT

DEMUX: $Y_i = D \cdot m_i(S_2, S_1, S_0)$ where m_i is the i -th minterm of the select lines. Decoder: $Y_i = m_i$ (data implicitly 1). Relationship: DEMUX = Decoder with enable=Data. Structurally: DEMUX adds one AND gate per output ($Y_i = EN_i \text{ AND } D$), or equivalently, connecting the data signal to the decoder enable input implements 1-to-8 DEMUX using a 3-to-8 decoder with no extra gates. In VLSI, they are identical circuits with different signal roles.

Q35

Explain how a 4-input LUT (Look-Up Table) is implemented as a 16-to-1 MUX with SRAM cells. Show the programming for $f(A,B,C,D) = \sum m(0,3,5,6,9,10,12,15)$ and explain how LUT cascading in modern FPGAs implements functions beyond 4 inputs.

KEY CONCEPT / APPROACH HINT

16 SRAM cells store truth table. 4 select lines are function inputs A,B,C,D. MUX routes selected SRAM cell to output. Programming: load truth table bits at positions 0,3,5,6,9,10,12,15 with 1, rest 0. SRAM bit-pattern = {1,0,0,1,0,1,1,0,0,1,1,0,1,0,0,1} (bit0=m0 etc). FPGA cascade: carry-chain LUTs share intermediate outputs; fracturable LUTs implement two 3-input functions simultaneously; routing matrix connects LUT outputs for multi-LUT functions.

Q36

Design a 4-bit bidirectional barrel shifter supporting both left and right logical shifts using 2-to-1 MUX networks. Calculate the minimum number of 2:1 MUXes required and describe the shift-amount encoding.

KEY CONCEPT / APPROACH HINT

Barrel shifter: shift by 0-3 positions in one combinational step. For 4-bit bidirectional: use 2 stages ($\log_2(4)=2$). Stage 1: shift by 0 or 1 (4 MUXes, controlled by S_0). Stage 2: shift by 0 or 2 (4 MUXes, controlled by S_1). Direction control: add one MUX stage to reverse bit order before and after for right shift, or route different connections. Total: 2 shift stages $\times$ 4 = 8 MUXes unidirectional. Bidirectional adds 4 MUXes for direction = 12 total 2:1 MUXes.

Q37

Formally define a priority MUX distinguishing it from a standard select-based MUX. For a 4-input priority MUX where I3 has highest priority, derive the complete Boolean expression for the output, implement with gates, and identify an application in digital arbitration.

KEY CONCEPT / APPROACH HINT

Priority MUX: output=highest-priority active input, not select-based. $f = I_3 + I_3' \cdot I_2 + I_3' \cdot I_2' \cdot I_1 + I_3' \cdot I_2' \cdot I_1' \cdot I_0$. This equals the OR-reduced prioritized chain. Gate implementation: 4 AND gates (for masked terms) + 4 OR gates in chain. Differs from standard MUX: no explicit select lines -- input activity determines selection. Application: interrupt controller selects highest-priority pending interrupt; bus arbiter grants bus to highest-priority requester.

Q38

A CMOS transmission gate MUX is used instead of a logic-gate MUX. Compare the two implementations in transistor count, propagation delay, power consumption, and signal integrity. Explain threshold voltage drop in NMOS-only pass gates and the solution.

KEY CONCEPT / APPROACH HINT

TG MUX: 2 transistors/input (NMOS+PMOS pair) + 2 for select complement. Logic gate MUX: 6+ transistors (NAND-based). TG advantage: lower transistor count, lower input capacitance, faster for small fan-in. Signal integrity: NMOS-only pass gate: passes strong 0 (pull-down) but weak 1 ($V_{DD} - V_{th}$ voltage drop). PMOS-only: weak 0. Transmission gate (NMOS||PMOS): both types in parallel -- PMOS handles high voltage, NMOS handles low: full rail-to-rail swing. Long TG chains still degrade: add static CMOS buffers every 4-5 stages.

SECTION 5

Comparators, Parity & Hazard Analysis

Q41–Q46

Q39

Design a 4-bit odd parity generator circuit. Derive the Boolean expression, implement with a 2-level XOR tree minimizing gate count, and trace a complete end-to-end error detection scenario in a serial transmission system showing receiver logic.

KEY CONCEPT / APPROACH HINT

Odd parity $P = A_3 \oplus A_2 \oplus A_1 \oplus A_0 \oplus 1$ (invert even parity). XOR tree: level 1 = $(A_3 \oplus A_2)$ and $(A_1 \oplus A_0)$: 2 XOR gates. Level 2 = XOR of level 1 outputs: 1 XOR gate. Invert for odd: 1 NOT gate. Total: 3 XOR + 1 NOT = 4 gates. Receiver: XOR all 5 received bits (4 data + 1 parity). Odd parity maintained $\rightarrow$ result=1 $\rightarrow$ no error. Result=0 $\rightarrow$ single-bit error detected. Cannot locate error position.

Q40

Compare even parity, odd parity, and Hamming(7,4) in terms of error detection and correction capability using the Hamming distance metric. Design the complete Hamming(7,4) encoder showing the parity bit computation logic using only XOR gates.

KEY CONCEPT / APPROACH HINT

Even/odd parity: $d_{\min}=2$, detects 1-bit, corrects 0. Hamming(7,4): $d_{\min}=3$, detects 2-bit errors, corrects 1-bit. Encoder: 4 data bits d_1, d_2, d_3, d_4 . Parity bits: $P_1 = d_1 \oplus d_2 \oplus d_4$, $P_2 = d_1 \oplus d_3 \oplus d_4$, $P_4 = d_2 \oplus d_3 \oplus d_4$. Transmitted order: $P_1, P_2, d_1, P_4, d_2, d_3, d_4$. Receiver syndrome: $s_1 = P_1 \oplus d_1 \oplus d_2 \oplus d_4$ etc. Non-zero syndrome identifies error position. Total: 3 XOR gates for encoder, 3 XOR gates for syndrome, 3-to-8 decoder for correction.

Q41

A 4-bit magnitude comparator compares A and B producing $A > B$ output. Derive the minimum gate-count Boolean expression using a hierarchical approach and identify the critical timing path. How many gate delays constitute the worst case?

KEY CONCEPT / APPROACH HINT

$A > B = A_3B_3' + EQ_3.(A_2B_2') + EQ_3.EQ_2.(A_1B_1') + EQ_3.EQ_2.EQ_1.(A_0B_0')$ where $EQ_i = XNOR(A_i, B_i)$. Gate structure: 4 XNOR (equality) + 4 AND for masking + 1 large AND-OR chain. Critical path: XNOR(bit 3) $\rightarrow$ EQ3 $\rightarrow$ AND $\rightarrow$ OR $\rightarrow$ output for bit-0 contribution. Depth: XNOR(1 level) + AND-chain(3 levels) + OR(1 level) = 5 gate delays in the worst case for 4-bit. General n-bit: $O(n)$ delay without lookahead.

Q42

Explain wired-OR (wired-AND in negative logic) using open-drain/open-collector outputs. Show how wired-OR implements a combinational OR without active gate logic and analyze the VLSI performance implications in terms of RC delay and fanout scaling.

KEY CONCEPT / APPROACH HINT

Open-drain: output transistor only pulls low; external pull-up resistor pulls high. Multiple open-drain signals on shared wire: wire=0 if ANY driver pulls low $\rightarrow$ wire-AND in positive logic (= wired-OR in negative logic). I2C uses this. RC delay: $t_{\text{delay}} = 0.69 * R_{\text{pullup}} * C_{\text{wire}}$. With n drivers: C_{wire} grows linearly with n. For n=10 drivers: delay 10x worse than single driver. Active OR gate: delay independent of fanout. Wired logic: useful for board-level bus sharing but slow -- avoid in high-speed VLSI datapath.

Q43

Design a combinational circuit detecting if a 4-bit BCD digit exceeds 4 (i.e., detects digits 5-9). Minimize the logic using a Karnaugh map with don't-cares and verify using the minimum SOP expression.

KEY CONCEPT / APPROACH HINT

Valid BCD 0-9. Detect inputs 0101 to 1001 as TRUE. Don't-cares: 1010-1111 (invalid BCD, 6 entries). K-map (4 variables ABCD, A=MSB): Group: A (covers 8,9 including don't-cares 10-15). Group: BC (covers 6,7 including don't-cares). Group: BD (covers 5,7 and don't-cares). Minimum SOP: $f = A + BC + BD = A + B(C+D)$. Verify: $f(0101)=0+1.(0+1)=1$ correct. $f(1001)=1$ correct. $f(0100)=0+1.(0+0)=0$ correct.

Q44

Describe Manchester carry chain logic as used in high-speed VLSI adders. Contrast it with static CMOS Full Adder in transistor count and delay, explaining the dynamic logic precharge mechanism and its sensitivity to charge sharing.

KEY CONCEPT / APPROACH HINT

Manchester carry: dynamic pass-transistor chain. Each cell: Generate transistor (G signal) pulls carry output high directly; Propagate transistor (P signal) passes carry-in to carry-out. Precharge phase: CLK=0, all carry nodes precharged high. Evaluate: CLK=1, generates/propagates resolve. Transistor count: ~4 per bit vs 28 for static complementary CMOS FA. Delay: faster carry path (2-3 FET delays vs 4-6). Risk: charge sharing between internal nodes can corrupt output if intermediate capacitances are comparable. Solution: add keeper transistors or use dynamic DOMINO style.

Q45

Identify the three types of hazards in combinational logic (static-1, static-0, dynamic). For the function $f = A'C + BC$, identify the static hazard using a K-map, specify the transition causing it, and derive the consensus term that eliminates it.

KEY CONCEPT / APPROACH HINT

Static-1 hazard: output should stay 1 but glitches to 0 momentarily. Static-0: stays 0 but glitches to 1. Dynamic: transitions more than once. For $f=A'C+BC$: K-map shows two groups not bridging. Hazardous transition: $A=0, B=1, C=1$ to $A=1, B=1, C=1$ (only A changes). Both minterms covered: $m(0,1,1)=A'BC$ and $m(1,1,1)=ABC$. During A's transition, neither term may be active simultaneously $\rightarrow$ static-1 hazard. Consensus term BC bridges both groups: $f=A'C+BC+BC\dots$ BC already present. Correct consensus: $A'B$ bridges the gap. Add $A'B$: $f=A'C+BC+A'B$ eliminates hazard.

Q46

How does a Programmable Logic Array (PLA) differ from a PAL (Programmable Array Logic) in its AND and OR plane programmability? Explain how PLA implements SOP and POS forms, and contrast both with ROM-based logic implementation for a 6-input, 4-output function.

KEY CONCEPT / APPROACH HINT

PLA: both AND plane (product terms) and OR plane (sum of products) are programmable. Flexible but complex. PAL: only AND plane programmable; OR plane is fixed (fixed-OR). Less flexible but faster and smaller. SOP in PLA: program AND plane for desired product terms, OR plane selects which products sum per output. POS: complement inputs drive AND plane, OR-AND structure. 6-input comparison: ROM=64 locations $\times$ 4 bits=256 bits; PLA=k product terms $\times$ (12 literals + 4 outputs) -- efficient when $k \ll 64$; PAL=k product terms per output (less sharing). ROM wins for irregular dense functions; PLA for sparse structured control.

SECTI
ON 6

Advanced Topics in Combinational VLSI

Q47–Q
50

Q47

Explain multi-level logic networks and why technology mapping from 2-level SOP to multi-level reduces VLSI area. Describe the algebraic factoring approach with a concrete example and identify the synthesis tool and algorithm traditionally used.

KEY CONCEPT / APPROACH HINT

2-level SOP: all minterms explicitly OR'd -- large fan-in gates, high literal count, high area. Multi-level: intermediate signals shared across outputs -- literal count reduced. Example: $f=ac+ad+bc+bd$ factors to $(a+b)(c+d)$ -- 4 literals vs 8, saves 2 gates. Algorithm: Algebraic division finds common sub-expressions. Tool: SIS (UC Berkeley) used divisor-quotient decomposition. Modern flows: ABC tool with AIG (And-Inverter Graph) optimization. Technology mapping: DAG nodes matched to cell library gates for minimum area/delay.

Q48

Distinguish between static and dynamic hazards in combinational logic and explain why complementary CMOS logic (pull-up PMOS network + pull-down NMOS network) inherently prevents static hazards that can occur in NMOS-only pseudo-NMOS or pass-transistor designs.

KEY CONCEPT / APPROACH HINT

Static hazard: momentary incorrect output during input transition while output should remain constant. Dynamic: output transitions more than once. NMOS-only: during input transition, both paths may conduct simultaneously (shoot-through) or neither may (floating output) -> glitches. Complementary CMOS: PUN and PDN are dual networks -- when PDN conducts, PUN does not, and vice versa (by duality). Output always driven from either rail: no floating, no shoot-through in steady state. Transition: only one brief moment of conflict during input change -- inherently glitch-suppressed by the dual network structure.

Q49

Compare ROM-based, PLA-based, and standard-cell netlist implementations for a 6-input, 4-output combinational function. Provide the area estimate for each approach and specify the technology and design constraints under which each implementation strategy wins.

KEY CONCEPT / APPROACH HINT

ROM: $2^6 \times 4 = 256$ bits. Any function, regular layout, $O(1)$ access after address decode. Area: large for small functions, fixed cost. PLA: k product terms $\times (12+4)$ bits. Area = $k \times 16$ bits. Wins when $k \ll 64$ (few product terms, high regularity). Standard cell: mapped netlist -- area proportional to logic complexity, highly optimized for well-structured functions (adders, decoders). ROM wins: large irregular truth tables, embedded in datapath. PLA wins: FSM next-state/output logic, microcode. Standard cell wins: arithmetic, structured logic, with synthesis optimization.

Q50

Prove that a 4-to-1 MUX with select lines S_1, S_0 can implement any Boolean function of three variables. Show the complete implementation for $f(A,B,C) = \sum m(0,2,5,7)$ identifying Shannon residues and MUX input assignments.

KEY CONCEPT / APPROACH HINT

Shannon expansion on A (S_1) and B (S_0): $f = A'B'.f(0,0,C) + A'B.f(0,1,C) + AB'.f(1,0,C) + AB.f(1,1,C)$.
 Compute residues: $f(0,0,C)$: minterms with $A=0, B=0$: $m_0=C', m_1=C$. $f(0,0,C) = C'(m_0) \rightarrow$ check:
 $(A,B,C)=(0,0,0)=m_0=1, (0,0,1)=m_1=0$. So $f(0,0,C)=C'$. $f(0,1,C)$: $m_2=(0,1,0)=1, m_3=(0,1,1)=0$.
 $f(0,1,C)=C'$. $f(1,0,C)$: $m_4=0, m_5=1$. $f(1,0,C)=C$. $f(1,1,C)$: $m_6=0, m_7=1$. $f(1,1,C)=C$. MUX: $I_0=C', I_1=C', I_2=C, I_3=C$. $S_1=A, S_0=B$. This proves any 3-variable function is implementable.

Q51

Explain the concepts of logic cone depth and critical path timing in VLSI synthesis. Describe how retiming transforms a circuit to balance cone depths and maximize F_{max} , with a concrete pipeline stage example.

KEY CONCEPT / APPROACH HINT

Logic cone: all combinational logic feeding a flip-flop output. Cone depth = longest gate-delay path = critical path. Clock period $T \geq \max_cone_depth + t_{setup}$. Retiming: legally moves flip-flops across combinational gates while preserving input-output behavior. Example: if stage A has depth 8ns and stage B has 2ns with a register between, retiming can move logic from A to B $\rightarrow$ both become 5ns $\rightarrow$ F_{max} doubles. Algorithm: solve LP or use Leiserson-Saxe retiming. Constraint: FFs cannot be moved through circuit boundaries. Modern EDA: Synopsys DesignCompiler retiming option performs this automatically.

Q52

What is Combinational Equivalence Checking (CEC) and how does the miter circuit construction enable formal verification between two combinational implementations? Compare SAT-solver-based CEC with BDD-based CEC in terms of scalability and practical use.

KEY CONCEPT / APPROACH HINT

CEC: formally proves two combinational netlists compute identical Boolean functions for ALL 2^n input combinations without simulation. Miter construction: connect both circuits to identical primary inputs, XOR corresponding outputs, OR all XOR outputs into single proof point. If miter output can be 1 (satisfiable) $\rightarrow$ circuits differ, and SAT solver provides counterexample. If UNSAT $\rightarrow$ circuits are equivalent. SAT-based CEC: handles 100k+ gate netlists, incremental learning, practical for industrial ECO flows. BDD-based: exact but suffers exponential blowup for arithmetic circuits. Modern tools (Cadence Conformal): use hybrid SAT+structural hashing for scalability.

Q53

Explain pass transistor logic (PTL) XOR gate implementation. Show how 6 transistors implement a 2-input XOR, identify the signal strength degradation problem in NMOS-only PTL, and describe the keeper/transmission gate solutions used in modern VLSI.

KEY CONCEPT / APPROACH HINT

PTL XOR: when $A=0$, pass B to output; when $A=1$, pass B' to output. Implementation: 2 NMOS pass transistors (select B or B' based on A) + pre-computed B' (needs inverter = 2 more transistors) + A complement = 6 transistors total. Signal degradation: NMOS pass transistor drives output to V_{DD} when input is 1, but threshold drop V_{th} reduces high level to $V_{DD}-V_{th}$. Cascaded stages: each stage loses V_{th} $\rightarrow$ eventual logic failure. Solution 1: keeper PMOS (weak pull-up feedback) restores to V_{DD} . Solution 2: transmission gate (NMOS+PMOS in parallel) -- PMOS conducts for high inputs, NMOS for low -- full rail-to-rail swing without V_{th} drop.

Q54

Model a combinational circuit as a Boolean network DAG and explain how this DAG is used during technology-independent optimization in a VLSI synthesis flow. Specifically describe the operations of node elimination, algebraic substitution, and Boolean resubstitution.

KEY CONCEPT / APPROACH HINT

DAG: directed acyclic graph where primary inputs = source nodes, logic gates = internal nodes, primary outputs = sink nodes, edges = signal dependencies. Technology-independent optimization operates on DAG: (1) Node elimination: if a node has few fanouts and small support, absorb into fanout cones -- reduces node count. (2) Algebraic substitution: find divisor D of node function F; replace F with quotient Q and remainder R using D -- reduces literal count if sharing beneficial. (3) Boolean resubstitution: use don't-care conditions (observability DCs) to simplify node function beyond algebraic methods. Tools: Berkeley SIS, ABC. Result: minimized multi-level network ready for technology mapping.

Q55

Explain stuck-at fault modeling in combinational circuits. Design a minimum test vector set achieving 100% stuck-at-0 and stuck-at-1 fault coverage for a 2-input AND gate (covering all 6 faults). Describe the D-algorithm for automatic test pattern generation.

KEY CONCEPT / APPROACH HINT

Stuck-at faults for 2-input AND gate: A_SA0 , A_SA1 , B_SA0 , B_SA1 , Out_SA0 , Out_SA1 = 6 faults. Test vectors: $\{A=1, B=1\} \rightarrow$ expected out=1; detects A_SA0 (out=0), B_SA0 (out=0), Out_SA0 (out=0). $\{A=1, B=0\} \rightarrow$ out=0; detects A_SA1 (out=1). $\{A=0, B=1\} \rightarrow$ out=0; detects B_SA1 (out=1). Out_SA1 : any input with expected 0 detects it. 3 test vectors achieve 100% coverage. D-algorithm: assigns D (good=1/faulty=0) or D' at fault site, propagates D to primary output through sensitized path (all other inputs set to non-controlling values), justifies line values backward to primary inputs. Systematic backtrack-based ATPG.

Q56

For $f(A,B,C,D) = \sum m(0,1,2,5,8,9,10)$ with don't-cares $d(3,7,15)$, apply the Quine-McCluskey tabular minimization method to identify all prime implicants and determine the minimum essential prime implicant cover.

KEY CONCEPT / APPROACH HINT

Step 1 - Group by 1-count: Group 0: {0000=0}. Group 1: {0001=1, 0010=2, 1000=8}. Group 2: {0101=5, 1001=9, 1010=10}. Group 3: {0011=3(DC), 0111=7(DC), 1111=15(DC)}. Step 2 - Combine adjacent groups (differ by 1 bit): 0,1->000-; 0,2->00-0; 0,8->-000; 1,3(DC)->00-1; 1,9->-001; 2,3(DC)->001-; 2,10->-010; 8,9->100-; 8,10->10-0; etc. Step 3 - Reduce: identify prime implicants that cannot be combined further. Step 4 - Essential PIs: 00-0 covers {0,2} uniquely (if no other PI covers m0 and m2 together). Final minimum cover typically: $A'B' + A'C'D' + AB'D' + A'BD$ (verify each minterm covered, don't-cares optionally used). Use Petrick's method for final minimization.